

Assignment 2 AIOps

Rohan Sai P. (DA24B049)

Repository: <https://github.com/Rohan-git-iitm/aiops-assignment2>

Question 1

1.1 and 1.2

Image	Disk usage	Content size (compressed)
Naive (single-stage)	2.19 GB	565 MB
Multi-stage	678 MB	145 MB
Reduction	69.0%	74.3%

1.3

Most of the saving comes from the base image: the naive image uses the full `python:3.11` image, its base layers add up to about 1.2 GB, including a 679 MB layer of gcc, C headers and `-dev` libraries, while the multi-stage runtime uses `python:3.11-slim` (about 174 MB of base layers and no compiler), so the full image stays in the builder stage. The builder also installs with `--no-cache-dir`, which avoids the 71 MB pip cache found in the naive image (its pip layer is 427 MB against the 367 MB venv copied into the runtime), and copying only `spam_app.py` and `model.joblib` leaves out the dataset, training scripts and notebook files. What is left in the 678 MB image is mostly the packages the API needs at runtime (scikit-learn, NumPy, SciPy, FastAPI and uvicorn).

Question 2

2.3

50 new messages were each sent twice to the API running in Docker Compose. The first call was a cache miss and the second a cache hit.

	Server time (median)	Client time (median)
Cache miss	1.050 ms	2.505 ms
Cache hit	0.187 ms	1.653 ms
Speedup	5.6x	34% faster

Server time is the time taken for a prediction inside the API: the Redis lookup, plus running the model on a miss. Client time is the total time from the notebook sending the request to getting the answer.

`docker compose logs api:`

```
api-1 | cache MISS (4.790 ms): 'WIN a FREE gift card now! Click here: wi'
api-1 | cache HIT (0.349 ms): 'WIN a FREE gift card now! Click here: wi'
api-1 | cache MISS (0.988 ms): 'Running a bit late for lunch, be there i'
api-1 | cache HIT (0.278 ms): 'Running a bit late for lunch, be there i'
```

2.4

Docker Compose lets us create a multi-container application (here the API and Redis) in one file, including how each image is built, its ports, its environment variables and a shared network, and then start it with one command. A single Dockerfile only describes how to build one image and leaves the wiring between containers to us. Compose works on a single host, while Kubernetes runs containers across a cluster of nodes, decides which node each pod runs on, and keeps restoring the state we want through self-healing, rolling updates and scaling.

Question 3

3.1

System pods already request 850m CPU on the control-plane node and 100m on the worker node, which leaves 1150m and 1900m free out of 2 CPUs each. With a CPU request of c per pod, $\lfloor 1150/c \rfloor + \lfloor 1900/c \rfloor$ pods can run at the same time:

CPU per pod	Control plane	Worker	Pods at once
1000m	1	1	2
900m	1	2	3
500m	2	3	5

If each pod asked for a full CPU, only 2 pods could run at once, because the system pods use part of the first node. With 500m per pod, 5 pods fit, so we chose parallelism 4. This runs the 8 shards in two equal rounds of 4. Choosing 5 would not help, since the 8 shards would still need two rounds (5 and then 3).

3.2

`kubect1 get pods -o wide` while the Job was running:

NAME	READY	STATUS	AGE	NODE
shard-validate-job-0-92m6k	1/1	Running	2s	minikube-m02
shard-validate-job-1-nqbvx	1/1	Running	2s	minikube

```
shard-validate-job-2-hdbzj 1/1 Running 2s minikube-m02
shard-validate-job-3-brw9c 1/1 Running 2s minikube
```

3.3

Shard	0	1	2	3	4	5	6	7
Invalid rows	195	140	218	187	160	153	210	152

Each pod prints its result in its log. We read these logs through the Kubernetes API, which works for every pod no matter which node it ran on. We did not use a shared volume because minikube stores a volume on the disk of one node only, so pods running on the other node could not reliably write their results to it.

3.4

We would raise parallelism to 8. A third node with 2 allocatable CPUs runs only a 100m networking pod, so it adds about 1900m of free CPU, bringing the total to $1150 + 1900 + 1900 = 4950\text{m}$, and at 500m per pod that fits $2 + 3 + 3 = 8$ pods at once. All 8 shards would then run at once, halving the time.

Question 4

4.2

`kubectl get pods` before and after deleting `spam-api-79965dd7d4-d95v4`:

Before:

NAME	READY	STATUS	RESTARTS	AGE
spam-api-79965dd7d4-d95v4	1/1	Running	0	17s
spam-api-79965dd7d4-l5s8x	1/1	Running	0	48s

After:

NAME	READY	STATUS	RESTARTS	AGE
spam-api-79965dd7d4-l5s8x	1/1	Running	0	64s
spam-api-79965dd7d4-sqc4r	1/1	Running	0	16s

The ReplicaSet controller (running inside kube-controller-manager) recreated the pod. The Deployment creates a ReplicaSet that is told to keep 2 pods running. The controller keeps counting the running pods with the `app=spam-api` label, and when we deleted one the count fell to 1, so it started a new pod to get back to 2, as the `SuccessfulCreate` event from `replicaset-controller` shows.

4.3

v2 adds a `version` field to `/healthz` response and was deployed with the image tag `spam-api:v2`.

During the update, 173 requests were sent to `/healthz` and all 173 returned 200 (115 answered by v1, 58 by v2), so there was no downtime.

If the API had been deployed as a Job, there would be no rolling update. A Job cannot be changed once it is created, so to move to v2 we would have to delete it and create a new one, which stops all pods at the same time and causes downtime. The Job would also never finish, since a web server has to keep running.

4.4

A Deployment keeps a service running all the time, for example, it keeps 2 copies of the spam API alive, replaces any pod that disappears and updates pods one at a time, which is what an API that serves users needs. A Job runs a task until it is done, for example, each shard validator in Q3 stops after checking its shard, and the Job ends once all 8 shards are done. If we swapped them, the validator would be restarted every time it finished, and the API would lose self-healing and rolling updates.